MODULE 4

LONG RANGE (LoRa) COMMUNICATION

OBJECTIVES

1) Understand Long Range (LoRa) modulation technology.
2) Understand LoRa communication, especially regarding to LoRaWAN.

REQUIREMENTS

1) Arduino IDE/PlatformIO,
2) LoRa Module/Cosmic LoRa Aurora V2 ESP32 LoRa Module,
3) ANTARES Account,
4) Sensor (optional),
5) Actuator (optional)

4.1 LoRa

LoRa (Long Range) is a spread-spectrum radio modulation technique for long-range communication with a low data rate and relatively low power consumption. LoRa operates in sub-GHz frequency bands whose regulations vary by region. This technology is suitable for transmitting small data packets in applications such as smart agriculture, smart cities, industrial monitoring, and environmental sensing. LoRa is not a network protocol; the network protocol commonly built on top of LoRa is LoRaWAN.

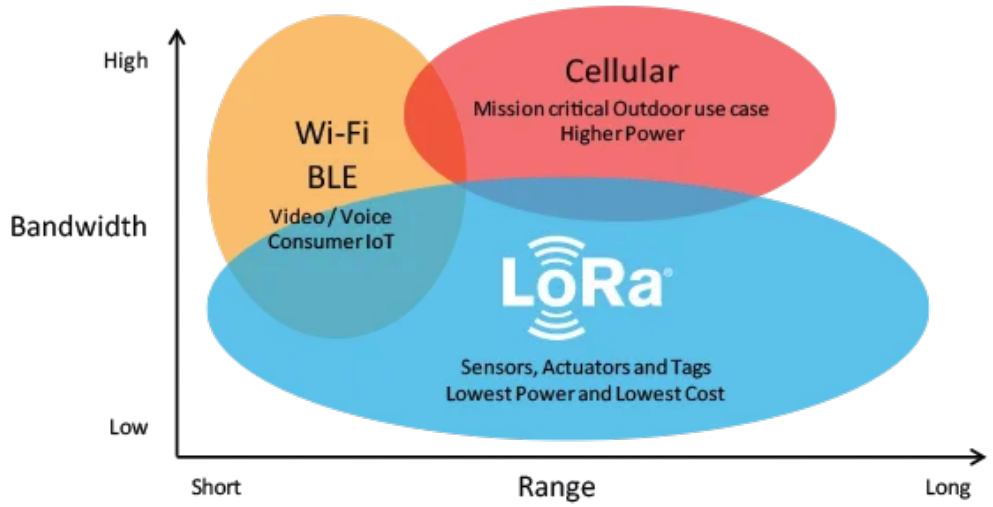


Figure 4.1: LoRa Bandwidth - Range

LoRa uses Chirp Spread Spectrum (CSS) and error-correction techniques to improve signal reception robustness. Parameters such as spreading factor, bandwidth, coding rate, frequency, and transmit power affect range, airtime, interference resistance, and data rate. A star-of-stars topology is a characteristic of the LoRaWAN architecture, not a mandatory property of peer-to-peer LoRa communication.

4.2 LoRa Communication Flow

LoRa communication uses frequency bands that comply with regional plans and local regulations. For example, LoRaWAN uses the EU863-870 regional parameters in parts of Europe and US902-928 in the United States. In Indonesia, the applicable regional parameters are AS923-2 in the **920–923 MHz** band. Users must ensure that frequency, transmit power, duty cycle, and device certification comply with applicable regulations.

The choice of frequency affects communication range and performance. End devices, such as IoT sensors or connected objects, use LoRa modulation to convert encoded data into chirp signals transmitted over the air. This modulation enables efficient long-range information transmission while conserving power.

LoRa uses **Chirp Spread Spectrum (CSS)** modulation, in which the chirp signal's frequency changes over time. In peer-to-peer communication, the transmitter and receiver communicate directly when they use compatible radio parameters, such as frequency, bandwidth, spreading factor, coding rate, and sync word. In a LoRaWAN network, a packet from an end device may be received by one or more gateways and then forwarded through an IP network to the network server. Gateways and servers are therefore components of LoRaWAN, not requirements for every form of LoRa communication.

4.3 LoRaWAN

LoRaWAN (Long Range Wide Area Network) is a network communication protocol built on LoRa modulation technology. LoRa defines how radio signals are transmitted over the air, whereas LoRaWAN

defines how devices join a network, transmit and receive data, perform authentication, maintain data security, and forward device data to user applications.

In general, LoRaWAN involves four main components:

a. **End device**: a sensor, actuator, or IoT device that collects and transmits data.
b. **Gateway**: receives LoRa packets from multiple **end devices** and forwards them through an IP network.
c. **Network server**: manages devices, validates packet security, removes duplicate data, and determines how data are forwarded.
d. **Application server**: processes and presents data for users or application systems.

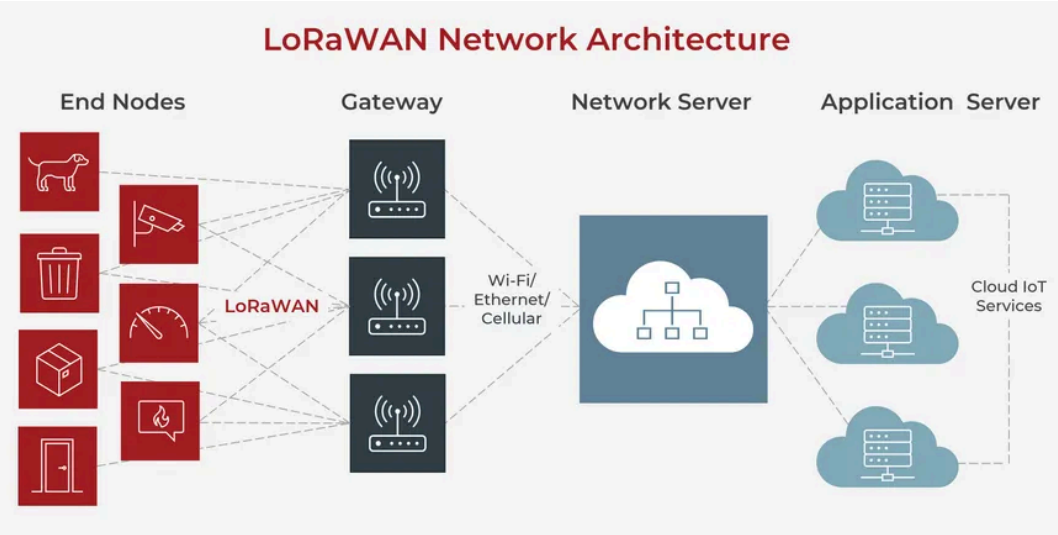


Figure 4.2: LoRaWAN Architecture

LoRaWAN uses a **star-of-stars** topology. In this topology, an **end device** is not bound to a particular gateway. A single data packet may be received by several gateways at once and then forwarded to the **network server**. The server selects and processes the appropriate packet and removes duplicates. This approach can improve communication range and reliability without requiring end devices to perform **routing** as they would in a **mesh** network.

LoRaWAN is suitable for applications that transmit small amounts of data periodically or in response to an event, such as temperature, humidity, air quality, asset location, water level, meter status, machine condition, and device status. It is often selected because it can provide wide-area coverage, low power consumption, support for many devices, and relatively low network deployment costs.

The LoRaWAN communication flow begins when an **end device** reads sensor data and then packages and encrypts the data according to the LoRaWAN format. The data are transmitted using LoRa modulation and may be received by one or more gateways. The gateways forward the packet through an internet connection or local network to the **network server**. After authentication, integrity checking, duplicate removal, and network management, the data are forwarded to the **application server** for storage, analysis, or dashboard presentation. For **downlink** communication, data travel from the **application server** back to the **end device** through the **network server** and a gateway.

LoRaWAN is appropriate when a system has the following characteristics:

a. Requires communication over distances of up to several kilometers;
b. Uses battery-powered devices that must operate for long periods;
c. Has many sensors distributed over a wide area;
d. Transmits only small amounts of data at specified intervals;

e. Does not require a high data rate or real-time communication;
f. Requires security, device authentication, and centralized network management.

Conversely, LoRaWAN is not well suited to applications that require video, audio, large-file transfer, very-low-latency control, or continuous data communication. Technologies such as Wi-Fi, Ethernet, cellular networks, or other higher-capacity communication protocols are more appropriate for those requirements.

4.4 LoRaWAN Device Classes

LoRaWAN devices are categorized into Class A, Class B, and Class C. These classes determine when a device opens downlink receive windows and therefore affect latency, power consumption, and communication patterns. Every LoRaWAN end device must support Class A; support for Class B and Class C is optional.

4.4.1 Class A

In Class A, uplink communication is initiated by the end device. After each uplink, the device opens two downlink receive windows. This pattern provides the lowest power consumption, but a downlink must wait until the device sends an uplink.

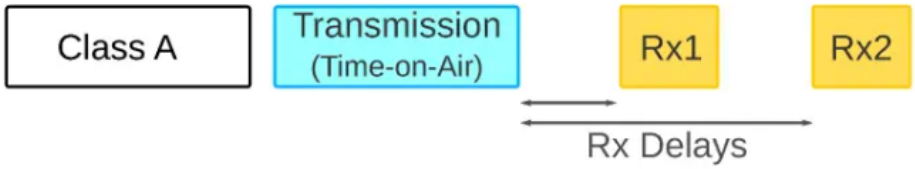


Figure 4.3: LoRa Class A Communication

4.4.2 Class B

Class B adds network beacons and scheduled ping slots to the Class A mechanism. With this schedule, the server has more predictable downlink opportunities, but power consumption and synchronization complexity are higher than in Class A.

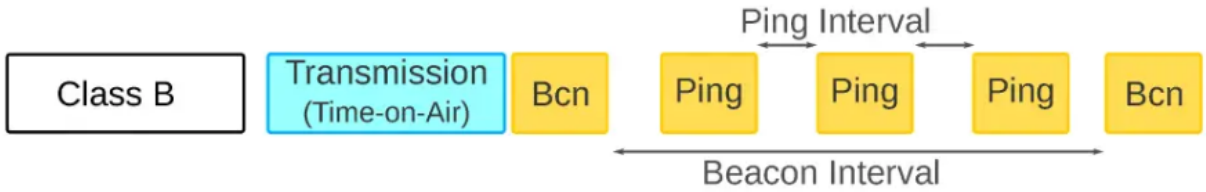


Figure 4.4: LoRa Class B Communication

4.4.3 Class C

Class C keeps the receiver open almost continuously, except while the device is transmitting. This provides the lowest downlink latency but the highest power consumption, so it is generally used for devices with a stable power supply.



Figure 4.5: LoRa Class C Communication

4.5 ANTARES

ANTARES is a horizontal Internet of Things platform developed by PT Telkom Indonesia Tbk. It provides services for connecting and managing IoT devices, storing sensor data, and presenting the data through dashboards or other applications. ANTARES supports various IoT implementations, including smart homes, smart metering, asset tracking, and smart buildings.

Figure 4.6: ANTARES Logo

ANTARES consists of two main service areas: the IoT platform and IoT connectivity. Its connectivity services support technologies such as LoRaWAN, NB-IoT, and GPRS, enabling IoT devices to transmit data to the ANTARES platform for monitoring and processing. It is the LoRaWAN IoT Platform that we will use in the implementation on the next section.

4.6 LoRa Protocol Implementation

4.6.1 Peer-to-peer LoRa using Cosmic LoRa Aurora V2

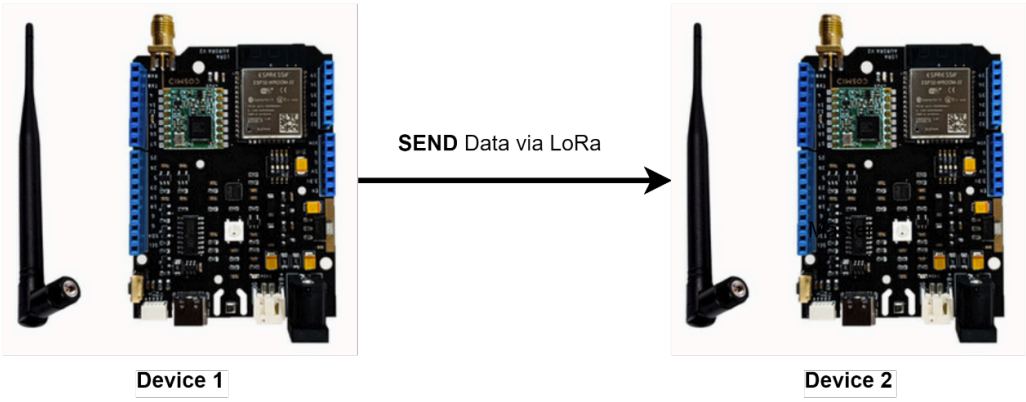


Figure 4.7: LoRa P2P Visualization

The Cosmic LoRa Aurora V2 is a LoRa development board that uses Cosmic's RFM95W LoRa transceiver, as shown in Appendix 4. The board uses an ESP32 as its microcontroller, so it provides both LoRa and Wi-Fi connectivity. It supports three power sources—USB, a LiPo battery, and a DC jack—and includes a smart power-switching system that can automatically select the active source. For example, while the LiPo battery is charging, the system can automatically switch to USB or the DC jack.

The LoRa library developed by Sandeep Mistry is used for the LoRa implementation with the Cosmic LoRa Aurora V2. It supports sending and receiving data through LoRa communication and is included with the `LoRa.h` header.

Information
The library is available from the following repository:
https://github.com/sandeepmistry/arduino-lora

4.6.1.1 LoRa Peer-to-Peer Source Code

The LoRa peer-to-peer transmitter and receiver source code is based on Cosmic's example, which is available at the following webpage:k

Source Code
The original code can be found in the link below:
https://github.com/cosmic-id/cosmic-lora-aurora/blob/main/examples/aurora-v2-lora-p2p-receiver/aurora-v2-lora-p2p-receiver.ino

- **Transmitter**

a. **Initialization**

LoRa Transmitter Initialization

```
// The receiver and transmitter use two libraries: SPI.h and Sandeep Mistry's LoRa.h
#include <SPI.h>
#include <LoRa.h>

// Define the pins used by the LoRa module
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;
```

To initialize LoRa P2P communication, the program uses the `SPI.h` library and Sandeep Mistry's `LoRa.h` library. It then defines the LoRa module pins in the `#define` declarations. The code above also initializes an additional variable, `counter`, whose value will be sent to the receiver.

b. **Setup**

LoRa Transmitter Setup

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Initialize the LoRa pin mode
  digitalWrite(LORA_EN, HIGH);  // Turn on the LoRa chip

  // Initialize the Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Configure the LoRa module pins
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Change the LoRa.begin(---E-) argument with a frequency permitted in your region
  // 433E6 for region Asia
  // 866E6 for region Europe
  // 915E6 for region North America
  // 920E6 for Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Set the sync word (0xF3) consistently on both transceivers
  // The sync word helps filter packets with different values
  // Valid range: 0x00–0xFF
  LoRa.setSyncWord(0xF3);

```

```
31    Serial.println("LoRa Initializing OK!");
  }
```

In the transmitter's `setup()` function, several settings must be defined:

- Declare the enable pin with **pinMode**. In this code, the enable pin is represented by `LORA_EN`, and its state is set to `HIGH`.
- Configure the LoRa module pins using `LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN)`.
- Start the LoRa module using `LoRa.begin(frequency)`, where `frequency` is set to a value permitted by regional regulations. This Indonesian practicum example uses `920E6` in the 920–923 MHz band (AS923-2). Ensure that the frequency and transmit power comply with regulations and device specifications.
- Set the same **sync word** on the transmitter and receiver. The sync word is a synchronization parameter, not a device address or security mechanism. Its value ranges from 0x00 to 0xFF and is set using `LoRa.setSyncWord(sync_word)`.

c. **Loop**

LoRa Transmitter Loop

```
1  void loop() {
2    Serial.print("Sending packet: ");
3    Serial.println(counter);
4
5    // Send a LoRa packet to the receiver
6    LoRa.beginPacket();
7    LoRa.print("hello ");
8    LoRa.print(counter);
9    LoRa.endPacket();
10
11   counter++;
12   delay(5000);
13 }
```

- **Receiver**

a. **Initialization**

LoRa Receiver Initialization

```
1  // The receiver and transmitter use two libraries: SPI.h and Sandeep Mistry's LoRa.h
2  #include <SPI.h>
3  #include <LoRa.h>
4
5  // Define the pins used by the LoRa module
6  #define LORA_NSS  15
7  #define LORA_RST  0
8  #define LORA_DIO0 27
9  #define LORA_EN   32
10
11 #define LORA_TX_POWER         20
12 #define LORA_SPREADING_FACTOR 12
13
```

The receiver and transmitter use the same pin and library initialization. However, the receiver code does not declare the additional `counter` variable used by the transmitter as outgoing LoRa data.

b. **Setup**

LoRa Receiver Setup

```
void setup() {
  pinMode(LORA_EN, OUTPUT);     // Initialize the LoRa pin mode
  digitalWrite(LORA_EN, HIGH);  // Turn on the LoRa chip

  // Initialize the Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Configure the LoRa module pins
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Change the LoRa.begin(---E-) argument with a frequency permitted in your region
  // 433E6 for region Asia
  // 866E6 for region Europe
  // 915E6 for region North America
  // 920E6 for Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Set the sync word (0xF3) consistently on both transceivers
  // The sync word helps filter packets with different values
  // Valid range: 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}
```

In the receiver's `setup()` function, pay particular attention to the frequency and sync word. The transmitter and receiver must use matching values. In this example, the frequency is 920E6 and the sync word is 0xF3.

c. **Loop**

LoRa Receiver Loop

```
void loop() {
  // Attempt to parse an incoming packet
  int packetSize = LoRa.parsePacket();

  if (packetSize) {
    // Receive a packet
    Serial.print("Received packet '");

    // Read the received packet
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Display the RSSI of the received packet
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

In the `loop()` function, the receiver accepts data from a transceiver that uses the same sync word. `LoRa.parsePacket()` checks for an incoming packet. When a packet is available, it is read with `LoRa.readString()` while `LoRa.available()` remains true. Finally, `LoRa.packetRssi()` returns the received packet's signal strength in dBm.

4.6.2 LoRaWAN Implementation using ANTARES

The communication process can be described as follows:

End Device → LoRaWAN Gateway → ANTARES Platform

The end device transmits data using LoRa modulation and the LoRaWAN protocol. The gateway receives the radio packet and forwards it to the ANTARES infrastructure through an Internet connection.

This implementation uses LoRaWAN Class A and Activation by Personalization or ABP. In ABP activation, the device uses a predefined device address and session keys without performing a join procedure each time it starts.

4.6.2.1 Creating an Application and Device in ANTARES

Before uploading the program, create an application and a device on the ANTARES console. The application and device are created using the following steps:

a. Sign in to the ANTARES console.

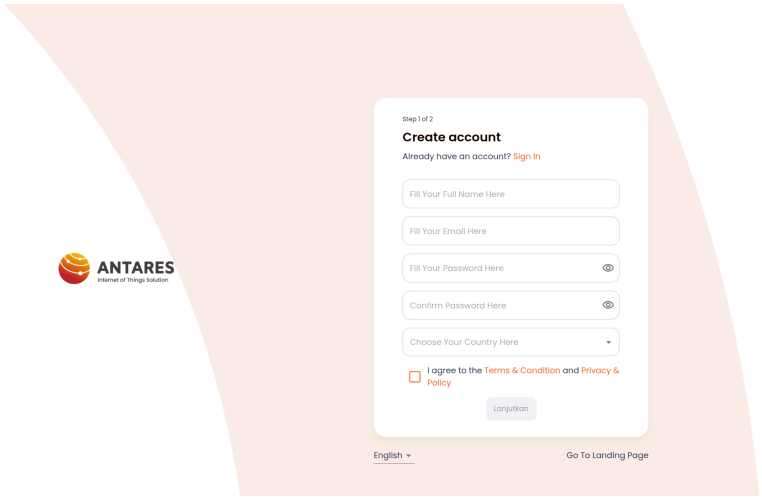


Figure 4.8: ANTARES Sign Up

b. Open the application page

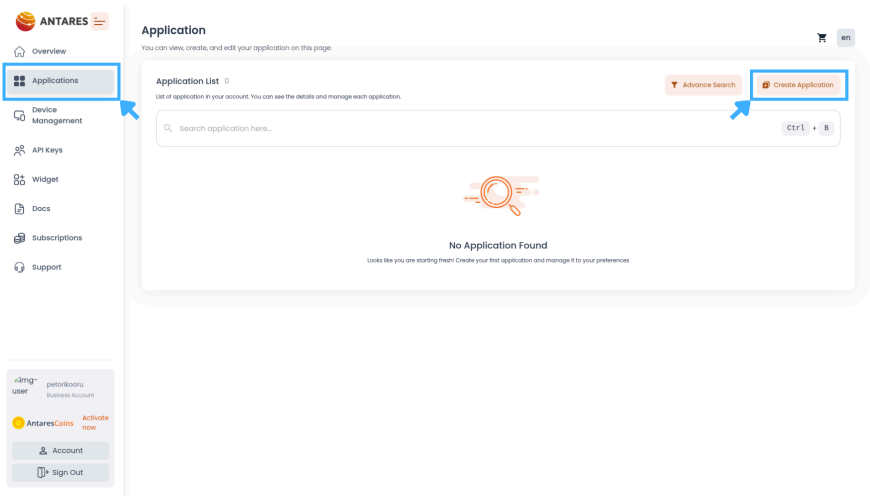


Figure 4.9: ANTARES Application Dashboard

c. Create a new application

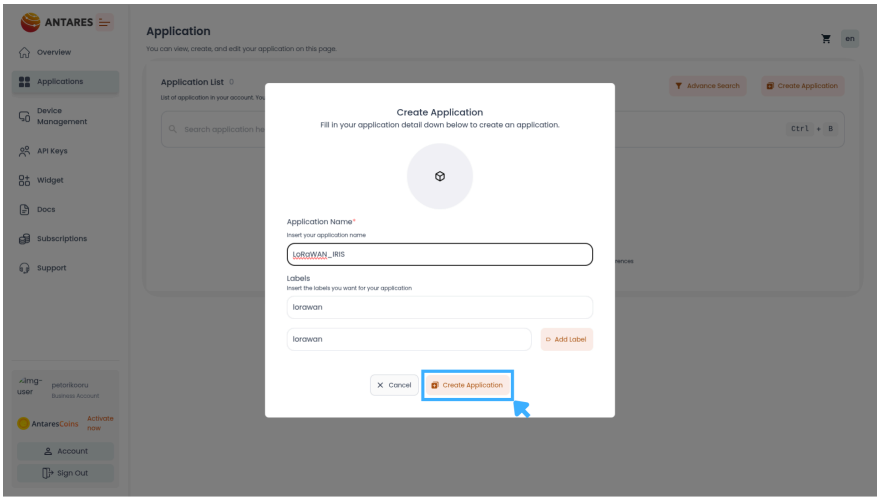


Figure 4.10: ANTARES Create Application Dialog

d. Then it will be redirected to device page. Create a new device.

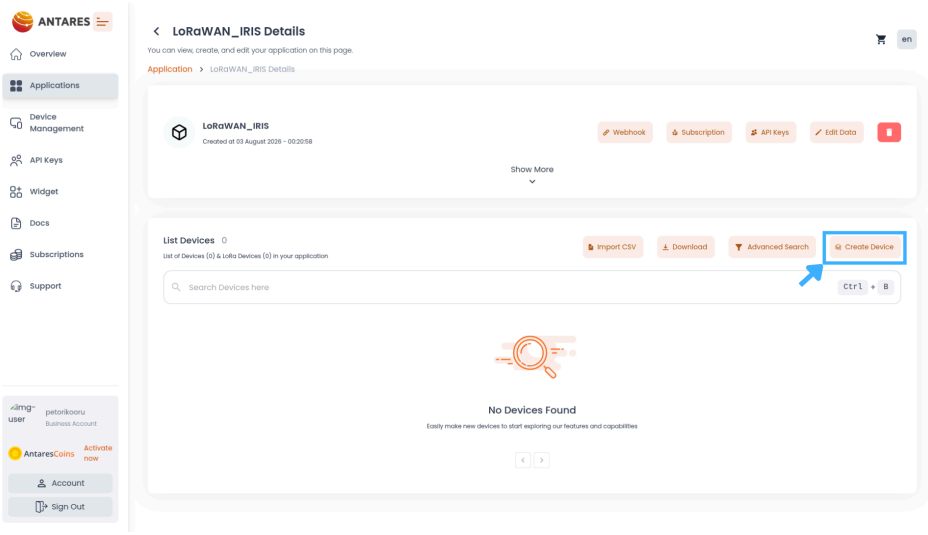


Figure 4.11: ANTARES Application's Device Page

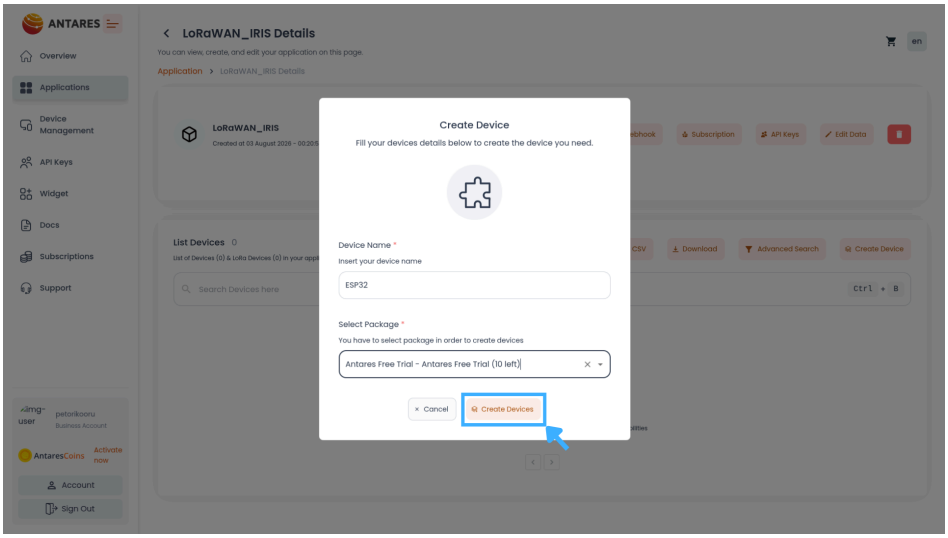


Figure 4.12: ANTARES Create Device Dialog

e. After that, it will be redirected to the device's detail. Select **Set LoRa**.

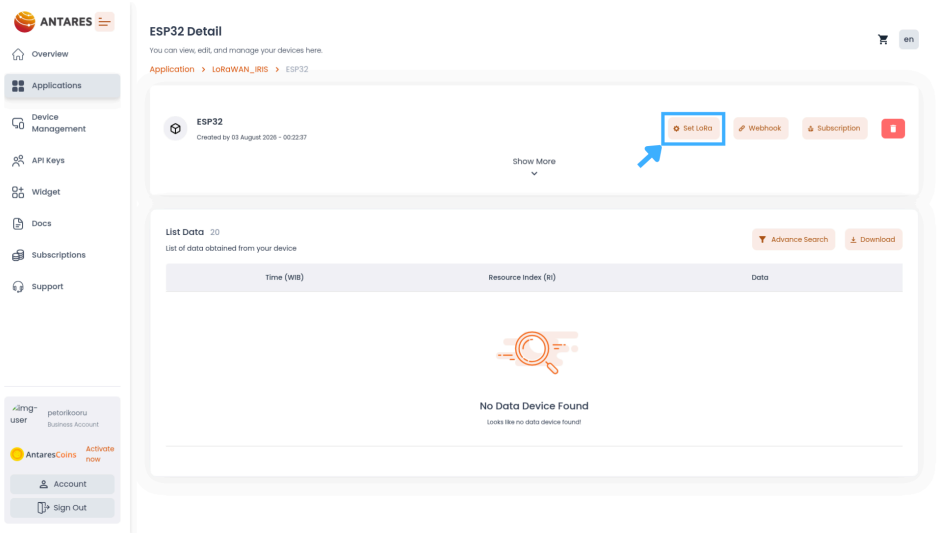


Figure 4.13: ANTARES Device's Detail

f. Configure the LoRa settings as below:

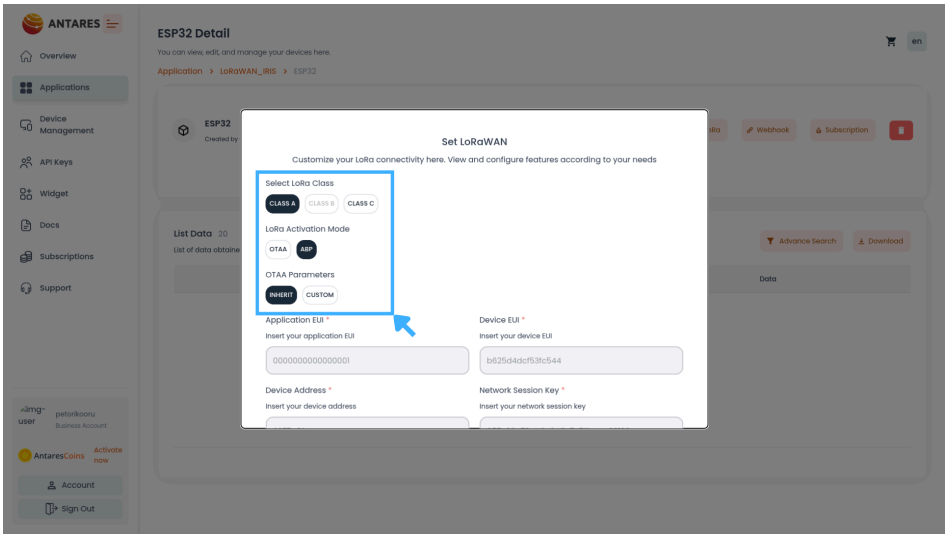


Figure 4.14: ANTARES Device's Detail

g. Finally, save the generated LoRaWAN parameters. You can take a look at the parameters needed from below figure.

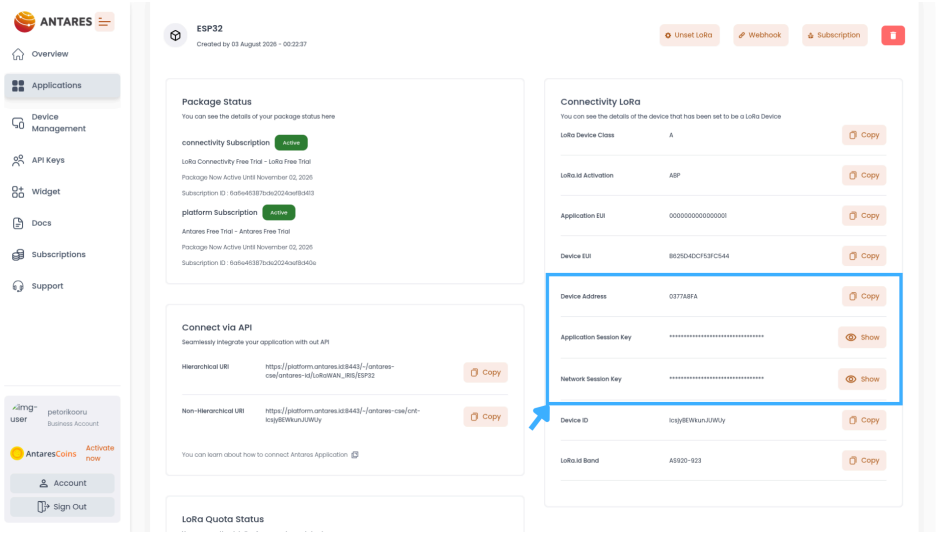


Figure 4.15: ANTARES Device's LoRaWAN parameter

ANTARES generates the following ABP parameters:

- **Device Address** or `DevAddr`;
- **Network Session Key** or `NwkSKey`;
- **Application Session Key** or `AppSKey`.

Copy and save these parameters before completing the device configuration. The parameters must be entered into the Arduino program.

Information
The network session key and application session key are confidential credentials. do not publish the actual keys in reports, screenshots, or public repositories.

4.6.2.2 Installing the ANTARES LoRaWAN Library

Install the ANTARES LoRaWAN library called `AntaresLoraID` in the Arduino IDE.

Information
The library is available from the following documentation:
https://docs.antares.id/pendahuluan/instalasi-library-arduino/antares-lorawan

The program uses the `lorawan.h` header file provided by the ANTARES LoRaWAN library.

4.6.2.3 Setting the ABP Parameters

Enter the Device Address, Network Session Key, and Application Session Key generated by ANTARES into the following variables:

ABP Parameters

```
const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";
```

Each value must be copied without spaces or additional characters.

4.6.2.3.1 LoRaWAN Source Code Program

The following program configures the Cosmic LoRa Aurora V2 as a LoRaWAN Class A end device using Activation by Personalization or ABP. The device sends counter data to ANTARES every 60 seconds and checks for downlink messages after each uplink transmission.

The program begins by including the SPI and ANTARES LoRaWAN libraries:

Required Libraries

```
#include <SPI.h>
#include <lorawan.h>
```

The `SPI.h` library manages communication between the ESP32 and the RFM95W radio module. The `lorawan.h` library provides the functions required to configure and operate the LoRaWAN connection.

The following definitions specify the RFM95W pin configuration used by the Cosmic LoRa Aurora V2:

LoRa Radio Pin Configuration

```
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18
```

GPIO 15 is used as the chip-select or NSS pin. GPIO 0 is connected to the reset pin, while GPIO 27 and GPIO 2 are connected to DIO0 and DIO1. GPIO 32 controls the radio-enable circuit and must be set to `HIGH` before the radio is initialized.

The ABP credentials generated by ANTARES are stored in the following variables:

ANTARES ABP Parameters

```
const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";
```

The `devAddr` variable contains the LoRaWAN device address. The `nwkSKey` variable contains the Network Session Key, while `appSKey` contains the Application Session Key. Replace the placeholder values with the parameters generated through the ANTARES console.

The following variables define the transmission interval and store the program state:

Program Variables

```
const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;
```

The `interval` value is expressed in milliseconds. A value of `60000` causes the device to send one uplink every 60 seconds. The `previousMillis` variable stores the time of the previous transmission, while `messageCounter` records the number of messages generated by the device. The `message` array stores the uplink message. The `receivedData` array stores downlink data received from ANTARES.

The RFM95W pin mapping is then passed to the LoRaWAN library:

RFM95W Pin Mapping

```
const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
  .DIO0 = LORA_DIO0,
  .DIO1 = LORA_DIO1,
};
```

The `RFM_pins` structure allows the library to communicate with the RFM95W radio using the correct ESP32 pins.

The `setup()` function initializes the Serial Monitor, radio-enable circuit, SPI interface, RFM95W module, and LoRaWAN parameters:

Setup Function

```
void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz:
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);

  // It will be available to multiple LoRaWAN channels
  lora.setChannel(MULTI);

  // The transmission power should comply with the applicable regional frequency
  // regulations and the requirements of the LoRaWAN network
  lora.setTxPower1(15);

  // ABP credentials
  // Because ABP uses predefined session parameters,
  // the device does not perform an OTAA join procedure before transmitting data.
  lora.setNwkSKey(nwkSKey);
  lora.setAppSKey(appSKey);
  lora.setDevAddr(devAddr);

  Serial.println("LoRaWAN Class A ABP configuration completed");
}
```

The `loop()` function controls periodic uplink transmission and downlink reception:

Main Loop

```
void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);

    Serial.print("Sending: ");
    Serial.println(message);

    // The message will be send here
    lora.sendUplink(message, strlen(message), 0, 5);

    messageCounter++;

```

```
    channel = lora.getChannel();

    Serial.print("Channel: ");
    Serial.println(channel);
  }

  lora.update();

  receiveStatus = lora.readData(receivedData);

  if (receiveStatus > 0) {
    bool isText = true;

    for (int i = 0; i < receiveStatus; i++) {
      if (!(
        (receivedData[i] >= 32 && receivedData[i] <= 126) ||
         receivedData[i] == 10 || receivedData[i] == 13)
        ){
        isText = false;
        break;
      }
    }

    if (isText) {
      Serial.print("Received text: ");

      for (int i = 0; i < receiveStatus; i++) {
        Serial.print(receivedData[i]);
      }
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>( receivedData[i]) < 0x10) {
          Serial.print('0');
        }

        Serial.print(static_cast<byte>( receivedData[i]), HEX);
        Serial.print(' ');
      }
    }

    Serial.println();
  }
}
```

The uplink message is created using the current message-counter value:

Creating the Uplink Message

```
snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);
```

For example, the first three generated messages are:

Generated Messages

```
LoRa Counter-0
LoRa Counter-1
LoRa Counter-2
```

The message is transmitted using the `sendUplink()` function:

Sending Data to ANTARES

```
lora.sendUplink(message, strlen(message), 0, 5);
```

The first parameter contains the message buffer. The second parameter contains the number of bytes to be transmitted. The third parameter determines whether the uplink is confirmed. A value of `0` selects an unconfirmed uplink. The fourth parameter specifies LoRaWAN application port `5`. After sending the message, the program increases the message counter and reads the channel used for transmission:

Counter and Channel Information

```
messageCounter++;

channel = lora.getChannel();

Serial.print("Channel: ");
Serial.println(channel);
```

The program then checks whether downlink data have been received:

Reading Downlink Data

```
receiveStatus = lora.readData(receivedData);
```

The return value indicates the number of bytes received. If the value is greater than zero, the program checks whether every byte represents a printable character.

When all received bytes are printable, the downlink is displayed as text:

Displaying a Text Downlink

```
Serial.print("Received text: ");

for (int i = 0; i < receiveStatus; i++) {
  Serial.print(receivedData[i]);
}
```

For example, an ANTARES downlink containing the text `ON` is displayed as:

Text Downlink Output

```
Received text: ON
```

If the received data contain non-printable bytes, the program displays each byte in hexadecimal format:

Displaying a Binary Downlink

```
Serial.print("Received hexadecimal data: ");

for (int i = 0; i < receiveStatus; i++) {
  if (static_cast<byte>( receivedData[i]) < 0x10) {
    Serial.print('0');
  }

  Serial.print(static_cast<byte>( receivedData[i]), HEX);
  Serial.print(' ');
}
```

A binary downlink may appear in the Serial Monitor as follows:

Binary Downlink Output

```
Received hexadecimal data: 01 FF A0
```

4.6.2.4 Receiving a Downlink

ANTARES can send a downlink message to the LoRaWAN device. Because the program uses Class A, the downlink is delivered through a receive window opened after an uplink. When a text downlink is received, the Serial Monitor displays output similar to:

Text Downlink Output

```
Received text: ON
```

When the downlink contains non-printable binary data, the program displays it in hexadecimal format:

Binary Downlink Output

```
Received hexadecimal data: 01 FF A0
```

If the device cannot send data to ANTARES, verify the Device Address, Network Session Key, Application Session Key, antenna connection, gateway coverage, radio pin configuration, device class, activation method, and LoRaWAN channel configuration.

APPENDIX: MODULE 4

P2P LoRa Transmitter Source Code

LoRa Transmitter

```
// Use two libraries: SPI.h and Sandeep Mistry's LoRa.h
#include <SPI.h>
#include <LoRa.h>

// Define the pins used by the LoRa module
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

int counter = 0;

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Initialize the LoRa pin mode
  digitalWrite(LORA_EN, HIGH);  // Turn on the LoRa chip

  // Initialize the Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Sender");

  // Configure the LoRa module pins
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Change the LoRa.begin(---E-) argument with a frequency permitted in your region
  // 433E6 for region Asia
  // 866E6 for region Europe
  // 915E6 for region North America
  // 920E6 for Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // Set the sync word (0xF3) consistently on both transceivers
  // The sync word helps filter packets with different values
  // Valid range: 0x00–0xFF
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  Serial.print("Sending packet: ");
  Serial.println(counter);

  // Send a LoRa packet to the receiver
  LoRa.beginPacket();
  LoRa.print("hello ");
  LoRa.print(counter);
  LoRa.endPacket();

  counter++;
  delay(5000);
}
```

P2P LoRa Receiver Source Code

LoRa Receiver

```
// Use two libraries: SPI.h and Sandeep Mistry's LoRa.h
#include <SPI.h>
#include <LoRa.h>

// Define the pins used by the LoRa module
#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_EN   32

#define LORA_TX_POWER         20
#define LORA_SPREADING_FACTOR 12

void setup() {
  pinMode(LORA_EN, OUTPUT);     // Initialize the LoRa pin mode
  digitalWrite(LORA_EN, HIGH);  // Turn on the LoRa chip

  // Initialize the Serial Monitor
  Serial.begin(115200);
  while (!Serial);
  Serial.println("LoRa Receiver");

  // Configure the LoRa module pins
  LoRa.setPins(LORA_NSS, LORA_RST, LORA_DIO0);

  // Change the LoRa.begin(---E-) argument with a frequency permitted in your region
  // 433E6 for region Asia
  // 866E6 for region Europe
  // 915E6 for region North America
  // 920E6 for Indonesia
  while (!LoRa.begin(920E6)) {
    Serial.println(".");
    delay(500);
  }

  LoRa.setTxPower(LORA_TX_POWER, PA_OUTPUT_PA_BOOST_PIN);
  LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);

  // The sync word helps filter packets with different values
  // Valid range: 0x00–0xFF (match the transmitter)
  LoRa.setSyncWord(0xF3);
  Serial.println("LoRa Initializing OK!");
}

void loop() {
  // Attempt to parse an incoming packet
  int packetSize = LoRa.parsePacket();

  // Receive a packet
  if (packetSize) {
    Serial.print("Received packet '");

    // Read the received packet
    while (LoRa.available()) {
      String LoRaData = LoRa.readString();
      Serial.print(LoRaData);
    }

    // Display the RSSI of the received packet (dB)
    Serial.print("' with RSSI ");
    Serial.println(LoRa.packetRssi());
  }
}
```

LoRaWAN Source Code

LoRaWAN Microcontroller Source Code

```
#include <SPI.h>
#include <lorawan.h>

#define LORA_NSS  15
#define LORA_RST  0
#define LORA_DIO0 27
#define LORA_DIO1 2
#define LORA_EN   32

#define SPI_MISO 23
#define SPI_MOSI 19
#define SPI_SCK 18

const char *devAddr = "YOUR-DEVICE-ADDRESS";
const char *nwkSKey = "YOUR-NETWORK-SESSION-KEY";
const char *appSKey = "YOUR-APPLICATION-SESSION-KEY";

const unsigned long interval = 60000;

unsigned long previousMillis = 0;
unsigned int messageCounter = 0;

char message[50];
char receivedData[255];

byte receiveStatus = 0;
int channel = 0;

const sRFM_pins RFM_pins = {
  .CS = LORA_NSS,
  .RST = LORA_RST,
  .DIO0 = LORA_DIO0,
  .DIO1 = LORA_DIO1,
};

void setup() {
  Serial.begin(115200);
  delay(2000);

  pinMode(LORA_EN, OUTPUT);
  digitalWrite(LORA_EN, HIGH);
  delay(100);

  SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_NSS);

  if (!lora.init()) {
    Serial.println("RFM95W radio was not detected");
    while (true) {
      delay(1000);
    }
  }

  Serial.println("RFM95W radio initialized");

  // A Class A device opens two receive windows after each uplink transmission.
  // Therefore, downlink messages are normally delivered after the device sends an uplink.
  lora.setDeviceClass(CLASS_A);

  // The data rate is configured using spreading factor 10 and a bandwidth of 125 kHz.
  // A higher spreading factor generally increases communication range and
  // receiver sensitivity but also increases airtime and reduces the data rate.
  lora.setDataRate(SF10BW125);

  // It will be available to multiple LoRaWAN channels.
```

```
  lora.setChannel(MULTI);

  // The transmission power should comply with the applicable regional frequency
  // regulations and the requirements of the LoRaWAN network.
  lora.setTxPower1(15);

  // ABP credentials.
  // Because ABP uses predefined session parameters,
  // the device does not perform an OTAA join procedure before transmitting data.
  lora.setNwkSKey(nwkSKey);
  lora.setAppSKey(appSKey);
  lora.setDevAddr(devAddr);

  Serial.println("LoRaWAN Class A ABP configuration completed");
}

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;

    snprintf(message, sizeof(message), "LoRa Counter-%u", messageCounter);

    Serial.print("Sending: ");
    Serial.println(message);

    lora.sendUplink(message, strlen(message), 0, 5);

    messageCounter++;

    channel = lora.getChannel();

    Serial.print("Channel: ");
    Serial.println(channel);
  }

  lora.update();

  receiveStatus = lora.readData(receivedData);

  if (receiveStatus > 0) {
    bool isText = true;

    for (int i = 0; i < receiveStatus; i++) {
      if (!(
        (receivedData[i] >= 32 && receivedData[i] <= 126) ||
        receivedData[i] == 10 ||
        receivedData[i] == 13
      )) {
        isText = false;
        break;
      }
    }

    if (isText) {
      Serial.print("Received text: ");

      for (int i = 0; i < receiveStatus; i++) {
        Serial.print(receivedData[i]);
      }
    } else {
      Serial.print("Received hexadecimal data: ");

      for (int i = 0; i < receiveStatus; i++) {
        if (static_cast<byte>(receivedData[i]) < 0x10) {
          Serial.print('0');
```

```
          }

          Serial.print(static_cast<byte>(receivedData[i]), HEX);
          Serial.print(' ');
        }
      }

      Serial.println();
    }
  }
```